Documento 01 · Guion del instructor

Para: instructor · DeepSeek Harness (dsh) · Generado el 30/10/2026

Guion del instructor — Del chat al agente: qué es un harness y cómo se usa

Duración: 150 minutos de núcleo, ampliable a 180. **Audiencia:** profesionales que ya usan chats o bots de IA; no se requiere experiencia previa en programación. **Formato:** sesión práctica. Cada alumno trabaja en su portátil. **Herramienta:** opencode con modelos gratuitos (sin clave ni cuenta), arrancado con `opencode web`.

Este guion está pensado para leerse en diagonal mientras se imparte. Lo que está en **negrita dentro de las secciones «Qué decir»** son las frases que conviene decir casi literalmente: son las que sostienen las analogías y evitan que el grupo se pierda.

1. Mapa de la sesión

#	Bloque	Min.	Acum.	Tipo
0	Bienvenida, diagnóstico y promesa	8	8	Charla
1	Del chat al agente	17	25	Charla
2	Anatomía de un harness	20	45	Charla + demo
3	Práctica 1 — Primer arranque y primer encargo	20	65	Práctica
—	Descanso	10	75	—
4	Permisos, sandbox y aprobaciones (Prácticas 2 y 3)	30	105	Práctica
5	Herramientas, contexto y memoria (Práctica 4)	22	127	Práctica
6	Práctica 5 — Encargo integrador	13	140	Práctica
7	Cierre, límites y evaluación	10	150	Charla

El bloque 4 es el corazón del seminario. Si vas mal de tiempo, recorta de cualquier sitio menos de ahí.

Si vas con retraso

Orden de recorte, de menos a más doloroso:

1. Bloque 5: deja MCP para una diapositiva y una frase, sin demo, y resume las partes 6 y 7 (ecosistema y datos) con la diapositiva «Mismas piezas, otras marcas» y la frase «revisa el programa, no solo el número».

2. Bloque 6: reduce a un encargo guiado hecho por todos a la vez con el instructor.
3. Bloque 2: acorta la demo del instructor a 3 minutos mostrando solo el panel de pasos.
4. Nunca recortes: bloque 4 completo, la Práctica 1 y el cierre.

Si vas sobrado (hasta 180)

Orden de ampliación:

1. Práctica 6 — Datos y modelo (20 min, opcional): análisis de un CSV de energía y un modelo lineal, con dos errores de registro puestos a propósito. Ideal para grupos técnicos (análisis de datos, ingeniería).
2. Bloque 6: de 13 a 25 minutos, con puesta en común de tres voluntarios.
3. Bloque 5: MCP con una demo real de conexión, o enseña el mismo encargo en Claude Code o Codex si tienes licencia.
4. Bloque 1: añade dos ejemplos más de tareas que el grupo reconozca.
5. Añade 5-10 minutos de preguntas al final.

2. Materiales que hay que tener listos

- Proyector y **letra grande** en el terminal y en el navegador. La mitad del grupo no verá una terminal a tamaño normal.
- Una **carpeta de práctica de ejemplo**, sin datos sensibles, con 5-10 archivos variados (un texto, una hoja de cálculo, alguna imagen) y sus subcarpetas.
- **Copia de seguridad** de esa carpeta, en otro sitio. El agente va a modificarla delante de todos.
- **Un plan B de modelo.** No hace falta clave, pero los modelos gratuitos pueden ir saturados: ten anotado otro gratuito de la lista y prueba el tuyo el día anterior.
- El resto de documentos del paquete: `03_laboratorio.md` (para los alumnos), `04_soluciones_y_errores.md` (para ti, abierto en otra ventana), `05_glosario_chuleta.md`, `06_evaluacion.md`.
- Regletas y alargadores. Doce portátiles no caben en cuatro enchufes.

Bloque 0 — Bienvenida, diagnóstico y promesa (8 min)

Objetivo: bajar la ansiedad, saber de dónde parte el grupo y dejar claro qué se llevan.

Qué decir

- Preséntate en un minuto y di a qué te dedicas con los agentes. Que se note que lo usas, no que lo has leído.
- Pregunta a mano alzada, tres preguntas rápidas: *«¿Quién usa un chat de IA a diario?»*, *«¿Quién le ha pedido algo y ha acabado haciéndolo él mismo porque el chat solo te decía cómo?»*, *«¿Quién ha oído hablar de agentes y no tiene claro qué son?»*. La tercera levanta casi todas las manos. Úsalo: **«Perfecto. Ese es exactamente el punto de partida de hoy, y en tres horas esto va a estar claro.»**
- Reparte el cuestionario inicial de `06_evaluacion.md`. **No se puntúa.** Dilo explícitamente: *«Esto no es un examen, es para que yo sepa por dónde empezar. Si no sabes una respuesta, escribe "no lo sé" y ya está.»* (3 min)
- La promesa, en cuatro puntos: al terminar **sabrás qué es un harness**; lo habrás **arrancado en tu propio portátil**; sabrás **qué permisos estás dando y por qué**; y sabrás **qué no conviene dejarle hacer**.
- Lo que NO vamos a hacer: no vamos a programar, no vamos a entrenar modelos, y no hace falta que sepas nada de informática más allá de usar el navegador y crear una carpeta.
- **Presenta los dos carriles** (un minuto): nadie necesita pagar, el curso se hace con opencode y modelos gratuitos; quien ya tenga Claude Pro o Max, o ChatGPT Plus o Pro, puede usar Claude Code o Codex (lección «Elige tu carril»).
- **Haz la «carrera de tres minutos»** con el mismo encargo para todos: primero a mano con un chat, luego con el agente. Que cada uno apunte cuánto ha tardado, y pídeles que anoten **minutos con chat** y **minutos con**

agente en la hoja de registro del laboratorio.

- Regla de la sala: **«Si te atascas más de tres minutos, levanta la mano. No hay nada que me haga perder más tiempo que diez personas atascadas en silencio.»**

Señales de que ha funcionado

El grupo responde a mano alzada y alguien dice en voz alta que no entiende la diferencia entre chat y agente. Eso es bueno: es la pregunta que abre el bloque 1.

Bloque 1 — Del chat al agente (17 min)

Objetivo: que entiendan el cambio de fondo, no la jerga.

Qué decir

Punto de partida: lo que ya saben hacer.

Todos sabéis usar un chat. Escribes una pregunta, te llega una respuesta de texto, y **tú** haces lo que haga falta con ella. Copias, pegas, abres el archivo, cambias el dato. El chat aconseja; el trabajo lo pones tú.

La analogía que sostiene todo el seminario:

Un chat es un consultor por teléfono: te dice qué hacer y lo haces tú. Un agente es un becario con acceso a tu ordenador: lo hace y tú revisas.

Di las dos mitades despacio y déjalas escritas en la pizarra. Vas a volver a esta frase cuatro veces más.

Tres diferencias, y solo tres:

1. **Tiene herramientas.** Puede leer y escribir archivos, ejecutar comandos, buscar en internet. No solo habla: toca cosas.
2. **Trabaja en bucle.** No da una respuesta y se calla. Da un paso, mira qué ha pasado, decide el siguiente. Diez pasos, cincuenta, los que hagan falta.
3. **Produce efectos reales.** Esto es lo importante y lo peligroso: al terminar, hay archivos cambiados en tu ordenador. No es una conversación, es un trabajo hecho.

El modelo y el harness.

Aquí está la palabra del título. **El modelo es el motor. El harness es el coche completo: el volante, los frenos, el salpicadero y las ruedas.** El motor solo no te lleva a ningún sitio. Y fíjate en la consecuencia: un chat también es un coche, pero uno muy sencillo — motor, un volante y nada más. Un agente es un coche con frenos, espejos y cinturón.

Ejemplo para que se vea la diferencia. Pide al grupo que imaginen un informe de veinte páginas en su escritorio.

- Con un chat: abres el documento, copias el texto, lo pegas en el chat, escribes «resúmeme esto», esperas, copias el resumen y lo pegas en otro archivo. Cuatro pasos tuyos.
- Con un agente: **«Resume el informe que está en esta carpeta y guárdame el resumen al lado, en un archivo nuevo.»** El agente abre el archivo, lo lee, escribe el resumen, te dice dónde lo ha dejado. Cero pasos tuyos, y un archivo nuevo que antes no existía.

La consecuencia incómoda, y hay que decirla hoy.

Un agente es como un becario muy motivado que trabaja a gran velocidad y no sabe lo que no sabe. Va a hacer lo que le pidas con una seguridad total, aunque se esté equivocando. Va a tocar archivos en segundos. Y si le has pedido algo mal, hará mal la cosa mala, rapidísimo. De ahí salen los frenos, y de ahí sale el bloque más importante de hoy.

Comprobación de comprensión

Pregunta a una persona concreta, no al aire: *«Si le pido a un agente que me organice las fotos del móvil, ¿qué hace exactamente?»* Busca que diga: mira los archivos, decide, mueve cosas. Si dice «me explica cómo hacerlo», vuelve a la analogía del teléfono.

Señales de que ha funcionado

Alguien pregunta «¿y si se equivoca?». Ya tienes el bloque 4 justificado: **«Exacto. Guarda esa pregunta, la contesto dentro de veinte minutos y es la parte más útil del seminario.»**

Bloque 2 — Anatomía de un harness (20 min)

Objetivo: que sepan nombrar las piezas. Sin esto, los permisos del bloque 4 son magia.

Qué decir

Un harness tiene seis piezas. Las vais a reconocer todas en la pantalla dentro de un rato.

Pieza	Qué es, en palabras llanas
Modelo	El motor. El que «piensa» y decide.
Herramientas	Las manos. Lo que puede hacer de verdad.
Bucle	El ritmo de trabajo: piensa, actúa, mira el resultado, repite.
Contexto	La mesa de trabajo. Todo lo que tiene delante ahora mismo.
Permisos	Los frenos. Qué puede hacer sin preguntar y qué no.
Interfaz	El salpicadero. Donde tú ves lo que está pasando.

El bucle, despacio, porque es lo que más se malinterpreta.

El modelo no ejecuta nada. El modelo propone. Piensa «para resumir este informe tengo que leerlo», y pide al harness que lea el archivo. El harness lo lee y le devuelve el contenido. El modelo lo lee, piensa el resumen, y pide que se escriba un archivo nuevo. Y así. **Piensa, actúa, observa, repite.** Cien veces si hace falta.

¿Por qué importa? Porque significa que **el harness está en medio de todo**. Es quien decide qué se le permite al modelo, y es quien ejecuta. El modelo no es el que toca tu ordenador: el que toca tu ordenador es el harness, siguiendo las órdenes de un modelo que nunca ve tu disco directamente salvo por lo que el harness le enseña.

Las herramientas. No hace falta memorizar la lista; lo que hace falta es saber que existe y saber mirarla:

- Leer archivos de la carpeta de trabajo.
- Escribir y modificar archivos.
- Ejecutar comandos en la terminal (esta es la más potente y la que más respeto merece).
- Buscar y leer en internet.
- Lanzar tareas en segundo plano mientras sigue con otra cosa.
- Preguntarte a ti cuando necesita decidir algo.
- Delegar una parte del trabajo en otro agente.
- Llevar una lista de tareas y marcar lo que va terminando.

El contexto: la mesa de trabajo. Aquí usa una imagen física: **imagina una mesa. Todo lo que el agente sabe en este momento está encima de la mesa: tu petición, los archivos que ha leído, lo que ha ido averiguando.** La mesa tiene un tamaño limitado. Cuando se llena de papeles, alguien tiene que recoger: se resume lo importante y se tira el resto. **Eso significa que en tareas muy largas el agente puede «olvidar» detalles del principio.** No es que sea tonto, es que ya no están en la mesa.

Consecuencia práctica que deben llevarse: una tarea larga y enrevesada es mejor partirla en dos sesiones que forzarla en una.

Los permisos, solo el titular. El becario trabaja dentro de una sala. Dentro hace lo que quiera. Para salir de la sala, te pide permiso. Eso es todo por ahora; lo vais a provocar vosotros en el bloque 4.

La interfaz. Insiste en esto porque es donde más se aprovecha la herramienta: **la interfaz no te enseña solo la respuesta final, te enseña los pasos.** Cada vez que el agente usa una herramienta, lo ves. Eso es tu ventana para darte cuenta de que va por donde no debe. **Si no miras los pasos, te enteras de los errores cuando ya están hechos.**

Demo del instructor (6-7 min)

Hazlo tú, en pantalla grande, narrando. No lo expliques antes: que lo vean y luego lo comentáis.

1. Abre el harness en la carpeta de práctica de ejemplo.
2. Escribe una petición de dos o tres pasos, por ejemplo: *«Mira qué hay en esta carpeta y hazme un índice de los archivos, agrupados por tipo, en un archivo llamado INDICE.md.»*
3. **Mientras trabaja, ve señalando en voz alta:** «está mirando qué hay — esta es la herramienta de leer», «ahora está escribiendo el archivo», «está comprobando que se ha escrito bien».
4. Al terminar, abre el `INDICE.md` delante de todos. Se ha creado un archivo que no existía. Ese es el momento «ajá» de este bloque: **«Nadie ha copiado ni pegado nada. Mirad la carpeta.»**

Señales de que ha funcionado

Preguntan por qué ha usado esa herramienta y no otra, o piden que lo repitas. Anota la duda y respóndela en el bloque 5.

Bloque 3 — Práctica 1: Primer arranque y primer encargo (20 min)

Objetivo: que cada alumno tenga el harness funcionando y haya completado un encargo de principio a fin.

Trabaja con `03_laboratorio.md` delante. Reparte el documento o pide que lo abran.

Secuencia

1. **Demuestra el arranque completo en tu portátil (4 min),** narrando cada paso: abrir la terminal, comprobar Node con `node -v`, comprobar opencode con `opencode --version`, entrar en la carpeta con `cd`, arrancar con `opencode web`, abrir el navegador, pulsar **Añadir proyecto**, abrir una **Nueva sesión**, elegir un modelo marcado como **«Gratis»** y escribir el primer mensaje.
2. **Todos a la vez (12 min).** Tú circulas. No te quedes con el primero que levante la mano: los que ya lo tienen, que ayuden al de al lado. Un alumno que explica arranca dos aprendizajes.
3. **Puesta en común (4 min).** Pregunta al grupo: *«¿Qué herramientas ha usado vuestro agente para hacer el resumen?»*. Que las nombren ellos.

Los tropiezos que vas a ver seguro

- **No encuentra la carpeta en «Añadir proyecto».** Que escriba la ruta a mano: `Documents/practica_agente`. Hasta que el proyecto no está añadido, el agente no tiene dónde trabajar.
- **El modelo está saturado o no responde.** Que elija otro modelo gratuito de la lista. Aprovecha para explicar que **el modelo es el motor, y aquí el motor lo pone gratis el proveedor a cambio de condiciones: algunos usan los datos que les envías**. Por eso, solo la carpeta de práctica.
- **Cambió `opencode.json` o creó una skill y no se nota.** opencode lee la configuración y las skills solo al arrancar: que lo apague con Ctrl+C y lo arranque de nuevo.

Qué decir mientras circulas

Nada de resolver por ellos con las manos en su teclado. Señala el botón y deja que lo pulse el alumno. **El objetivo es que sepan hacerlo sin ti el lunes.**

Señal de que ha funcionado

Todos tienen una sesión con al menos un encargo terminado y un archivo nuevo en su carpeta.

Descanso (10 min)

Antes de salir, avisa: **«Al volver vemos lo más importante de todo: los frenos. Y los vais a tocar vosotros.»** Que nadie se vaya pensando que lo bueno ya ha pasado.

Bloque 4 — Permisos, sandbox y aprobaciones (30 min)

Objetivo: que entiendan, y hayan sentido, qué se le permite al agente y quién decide.

Este bloque es la razón por la que el seminario existe. Los alumnos ya saben que un agente puede hacer cosas. Ahora tienen que aprender a dormir tranquilos mientras las hace.

Parte 1 — Los tres conceptos (8 min)

Son tres cosas distintas que la gente mezcla siempre. Escríbelas separadas en la pizarra:

Concepto	Qué es	Analogía
Sala (carpeta de trabajo)	La zona donde el agente puede moverse: la carpeta que abriste como proyecto. En opencode es una regla de permisos, no un muro: no hay sandbox que aísle el programa.	La sala donde trabaja el becario.
Política de permisos	El ajuste general: qué se permite sin preguntar y qué no.	El reglamento de la sala.
Aprobación	El aviso puntual que aparece cuando algo se sale de lo permitido.	El «¿seguro?» en la puerta.

Dentro de la sala, el becario hace lo que quiera. Para salir de la sala, te pide permiso.

Y añade la pieza que casi nadie espera: **tú puedes elegir el tamaño de la sala.** Cuanto más pequeña y más específica sea la carpeta de trabajo, menos avisos y menos riesgo. Esa decisión es tuya, y es la más importante que vas a tomar hoy.

Parte 2 — Práctica 2: Provocar una aprobación (10 min)

Que lo hagan todos a la vez, con instrucciones exactas del laboratorio. El objetivo es **ver el aviso y entender**

qué está pidiendo, no aprobarlo rápido.

Cuando aparezca el aviso, para la clase entera y léelo en voz alta, tú. Y lanza las tres preguntas:

1. **«¿Qué os está pidiendo exactamente?»** — Leer un archivo de fuera de la carpeta. Escribir en el escritorio. Ejecutar un comando. Que lo digan con sus palabras.
2. **«¿Qué pasaría si decís que sí?»** — Se hace, de verdad, ahora. No es una simulación ni una confirmación de cortesía.
3. **«¿Se puede acotar?»** — Casi siempre sí: pedirle que trabaje solo en tal carpeta, o que te enseñe antes lo que va a hacer.

El momento clave: «Permitir una vez» es solo esa vez. No es un permiso permanente y no se recuerda. Di la frase completa:

«Permitir una vez es permitir esa vez. La próxima te vuelve a preguntar. Es incómodo a propósito: ese pequeño fastidio es lo que te obliga a pensar si de verdad quieres que salga de la sala.»

Aquí aparecen dos comportamientos que hay que desactivar ya:

- **Aprobar por inercia.** Si te descubres pulsando «permitir» sin leer, para. Estás firmando sin leer.
- **Subir el nivel de permiso para que deje de preguntar.** Es como quitar el freno porque el coche hace ruido al frenar. Si algo se repite mucho, la solución no es aprobar cien avisos: **es mover el trabajo dentro de la carpeta de trabajo**, donde no hace falta permiso.

Regla de oro, y que la copien: si no entiendes qué te está pidiendo, no lo apruebes. Pídele que te lo explique con palabras, o que te diga qué va a tocar antes de tocarlo.

Si alguien usa Claude Code u otro agente con aprobación automática (por ejemplo, el **modo Auto**), no verá ningún aviso: el archivo se creará solo. Aprovéchalo delante de todos: **sin aviso, la decisión la ha tomado otra IA, no la persona.** Que cambie a modo manual (`Shift+Tab` en Claude Code) y repita.

Parte 3 — Práctica 3: Cambiar de política (8 min)

La misma tarea que antes, pero hecha dentro de la carpeta de trabajo. Todos la repiten y observan: **ya no pregunta nada.** Es la política de serie de opencode: dentro de la carpeta lee, escribe y ejecuta órdenes sin preguntar.

Después cambian la política: piden al agente que cree `opencode.json` en la raíz con `{"$schema": "https://opencode.ai/config.json", "permission": {"edit": "ask", "bash": "ask", "external_directory": "ask"}}`, **reinician** opencode (Ctrl + C y `opencode web`, sesión nueva) y repiten con `prueba_dentro2.txt`: **ahora sí pregunta**. El tropiezo típico es no reiniciar: la política se lee al arrancar. Al terminar, que borren `opencode.json` y reinicien.

Que saquen ellos la conclusión. Si no sale, dila tú:

«El agente no es más listo ni más tonto. Habéis cambiado sus reglas. Hay dos palancas y las dos son vuestras: dónde trabaja (la carpeta) y qué puede hacer sin preguntar (la política).»

Parte 4 — Cierre del bloque (4 min)

Tres ideas para que se queden:

- **El sandbox reduce el riesgo, no lo elimina.** Todo lo que está dentro de la sala autorizada, el agente lo puede tocar. Si dentro de la sala hay algo que no debería tocarse, el problema es lo que hay en la sala, no el agente.
- **La aprobación es tuya, no del sistema.** El aviso no decide: te informa. La responsabilidad es de quien pulsa.
- **Copia de seguridad antes de empezar.** La regla es aburrida y por eso se salta: **si no tienes copia de algo, no lo pongas en la carpeta de trabajo.**

Y una cuarta, en un minuto, con la diapositiva «El riesgo que no se ve»: **lo que el agente lee de fuera puede traer órdenes escondidas** (inyección de instrucciones). Afecta a todos los agentes, también a Claude y a ChatGPT. Regla práctica: **nunca juntes datos privados, contenido de fuera y salida a internet; quita al menos una.**

Señales de que ha funcionado

Sabrán decirte, sin mirar, la diferencia entre sandbox y aprobación, y por qué «permitir una vez» no vale para

siempre.

Bloque 5 — Herramientas, contexto y memoria (22 min)

Objetivo: que sepan qué puede hacer su agente, qué hacer cuando la conversación se degrada, y cómo no repetirse.

Parte 1 — Las herramientas, con calma (5 min)

Vuelve a la lista del bloque 2, ahora mirando la pantalla real: dónde se ve qué herramientas hay disponibles y qué está usando en cada paso. **La conclusión que buscamos: no hace falta memorizar nada, hace falta mirar los pasos.**

Y un aviso honesto: **la herramienta de terminal es la más potente de todas.** Es la que permite instalar programas, mover cosas y deshacer con la misma facilidad con la que rompe. Cuando el agente use la terminal, mira más despacio que cuando lee un archivo.

Parte 2 — El contexto, en la práctica (5 min)

Retoma la mesa de trabajo. Señales de que se está llenando: el agente repite cosas que ya hizo, olvida instrucciones que le diste al principio, o da vueltas sobre el mismo problema. Qué se hace:

- **Cerrar y abrir una conversación nueva**, diciendo en el primer mensaje lo que ya está hecho y dónde. Es más rápido que insistir.
- **Dividir el trabajo** en trozos con un resultado claro cada uno.
- **No arrastrar** una conversación larga para una tarea nueva y distinta. Cada tarea, su conversación.

Parte 3 — Memoria: lo que sobrevive (5 min)

Dos niveles, y la diferencia importa:

- **La conversación** — se acaba. Si cierras y abres otra, el agente no se acuerda de nada.
- **Lo permanente** — sobrevive a todo. Hay dos formas: las **notas** (archivo `AGENTS.md`) y los **procedimientos guardados** (skills).

  `AGENTS.md` son las notas que dejas al becario antes de que empiece. Si tienes una preferencia que se repite — «responde en español», «no toques la carpeta de facturas», «los informes van siempre en la carpeta `informes`» — no la repitas cada día: escríbela una vez. Hay notas **globales** (para todos tus proyectos) y notas **por proyecto**.

  Las skills son procedimientos guardados, como recetas de cocina. Cuando repites una tarea siempre igual, la escribes una vez con sus pasos y el agente la sigue cuando le dices «hazme el informe mensual». En la configuración de ejemplo hay cuatro de verdad, en `.opencode/skills/`, por ejemplo `resumen-informes`.

  Lección para que se la lleven: si te descubres explicando lo mismo por tercera vez, el problema no es el agente: es que no le has dejado notas.

Parte 4 — Práctica 4: Enseñarle un procedimiento (5 min)

Que cada alumno deje una instrucción permanente (una nota o una skill sencilla) y **compruebe en una conversación nueva que se respeta**. El detalle que hay que subrayar: **hay que probarlo en una conversación nueva.** En la misma conversación no demuestra nada, porque se acuerda por el contexto.

Parte 5 — MCP, si hay tiempo (2-3 min)

Solo el concepto, sin tecnicismos: **MCP es un enchufe estándar, como el USB.** Existen muchísimos programas que ya saben hacer cosas — GitHub, bases de datos, gestores de incidencias, servicios de memoria — y gracias a ese enchufe común se pueden conectar al agente y aparecen como herramientas nuevas. **Cuándo merece la pena: cuando la información que necesitas no está en archivos, sino en otro programa.**

Aviso de seguridad, obligatorio: **cada enchufe es una conexión nueva a tus cosas. Instalar un plugin o conectar un servidor es una decisión de confianza, igual que instalar un programa.** Si vas a conectar algo que toca datos reales de la empresa, consúltalo antes.

Las partes 6 y 7 no caben en los 22 minutos del bloque. Salen del margen de ampliación (hasta 180 minutos). En la sesión de 150, aplica el recorte nº 1: una diapositiva y una frase para cada una.

Parte 6 — Más allá de opencode: Anthropic y OpenAI (3 min, del margen)

La pregunta va a salir seguro: «¿y esto sirve para Claude o para ChatGPT?». Respuesta: **sí, mismas piezas, otro salpicadero.** Recuérdales el carril B: con Claude Pro o Max, o ChatGPT Plus o Pro, ya tienen Claude Code o Codex, y las notas (`AGENTS.md`) y las skills que escriban hoy les sirven allí casi sin cambios. Enseña la tabla «Mismas piezas, otras marcas»: Claude Code y Codex son el agente con código (terminal, escritorio, web, IDE); Claude Cowork y ChatGPT Work son el agente de oficina sin terminal; el análisis de datos dentro del chat es el paso intermedio, que no toca tu ordenador.

Después, la tabla de los frenos: `CLAUDE.md` o `AGENTS.md` son las notas; los modos *Plan*, manual, *Auto* y el sandbox son la sala del becario con otros nombres. **Los modos automáticos reducen avisos, no riesgos.** Y un dato que sorprende: en Claude Code el sandbox viene desactivado por defecto.

Datos a 30 de octubre de 2026. Antes de cada edición, repasa la documentación oficial: los nombres cambian cada pocos meses.

Parte 7 — Datos, modelos e ingeniería (3 min, del margen)

El agente no te da un número: te da un programa que calcula el número. Revisa el programa, no solo el número. Tres ideas: que describa los datos antes de calcular (filas, unidades, huecos); que separe datos de ajuste y de comprobación y diga el rango de validez del modelo; y que el cálculo de ingeniería lo firma una persona, con la norma vigente delante. Si hay tiempo, enlaza con la Práctica 6.

Bloque 6 — Práctica 5: Encargo integrador (13 min)

Objetivo: que hagan solos un encargo completo, y sobre todo que **verifiquen**.

Qué decir

Aquí ya no hay instrucciones mías. Piensa en una tarea pequeña y real tuya, que tenga que ver con archivos, y que puedas comprobar en dos minutos.

El patrón de un buen encargo

Escríbelo en la pizarra y que lo copien; es lo más útil que se llevan hoy:

1. Qué quiero (el objetivo, con un resultado concreto). **2. Dónde** (la carpeta exacta). **3. Qué NO tocar** (los límites, dichos explícitamente). **4. Cómo sabré que está bien** (tu criterio de éxito).

Ejemplo completo, para que se vea el patrón entero:

«En la carpeta `practica_agente/informes` hay cinco informes en texto. Léelos y créame `resumen_general.md` con una tabla de tres columnas: tema, conclusión y nivel de urgencia. **No modifiques ninguno de los cinco archivos originales.** Al terminar, dime qué archivos has leído y cuántas filas tiene la tabla.»

Verificación obligatoria

Y ahora la parte que la gente se salta: abrid el resultado y comprobadlo. No vale «tiene buena pinta». Cuenta las filas, mira si los cinco informes están, comprueba que los originales siguen intactos.

«El agente no firma su trabajo. Lo firmas tú cuando dices que está bien.»

Puesta en común (si vas sobrado)

Tres voluntarios, un minuto cada uno: qué pidieron, qué les devolvió el agente y **qué comprobaron**. Insiste en la tercera parte: lo interesante es lo que revisaron, no lo que pidieron.

Bloque 7 — Cierre, límites y evaluación (10 min)

Los tres errores que va a cometer todo el mundo

1. **Aprobar por inercia.** El aviso se convierte en un clic automático y deja de protegerte.
2. **Confiar sin verificar.** El resultado llega bien maquetado y con seguridad en el tono. Eso no es prueba de que sea correcto.
3. **Darle lo que no debería ver.** Si arrastras una carpeta con datos de clientes «para probar», ya está dentro de la sala.

Límites, dichos con honestidad

No endulces esto. Es información que protege a la gente:

- **opencode cambia muy rápido.** Habrá novedades que rompan lo anterior. No construyas nada crítico encima sin plan B.
- **De serie confía mucho en el agente.** Dentro de la carpeta lee, escribe y ejecuta órdenes sin preguntar; los frenos los pones tú en `opencode.json`.
- **No tiene sandbox.** Los permisos y las aprobaciones reducen el riesgo, pero no garantizan aislamiento y **no deben ser tu único control de seguridad**. Si el trabajo es delicado, usa una máquina virtual o un entorno dedicado y desechable.
- **Gratis no es privado.** Lo que el agente lee se envía al proveedor del modelo, y algunos modelos gratuitos usan esos datos para mejorar.
- **Lo que sí está en tu mano:** trabajar con el mínimo privilegio necesario, tener copias de seguridad de lo que el agente puede tocar, no exponer credenciales, y revisar plugins y comandos antes de permitirlos.

Las cuatro reglas para el lunes

1. **Copia de seguridad** de la carpeta antes de empezar.
2. **Una carpeta específica** para cada trabajo. Ni el escritorio entero ni, mucho menos, todo el disco.
3. **Nada confidencial ni sensible** en la carpeta de trabajo, salvo que aceptes el riesgo conscientemente.
4. **Revisa antes de aprobar y verifica al terminar.** Siempre. Las dos veces.

Evaluación y cierre (3-4 min)

Reparte el cuestionario final de `06_evaluacion.md` y la encuesta de satisfacción. Recuerda que no es un

examen.

Cierra con dónde seguir y con la frase que resume el día:

«Hoy no habéis aprendido a usar un programa. Habéis aprendido a dirigir a alguien que trabaja muy rápido, que no sabe lo que no sabe, y que tiene acceso a vuestros archivos. La herramienta os va a servir; los frenos os van a proteger.»

Material para seguir: la documentación de opencode (https://opencode.ai/docs), el glosario y la chuleta de `05_glosario_chuleta.md`.

Anexo — Chuleta de emergencia del instructor

Pasa esto	Di esto
Un alumno no arranca y se queda atrás	«Sigue conmigo sin arrancar, ahora paso y lo vemos. Mientras, apunta en qué paso se ha quedado.»
La interfaz va lenta y el grupo se impacienta	«Está pensando y trabajando. Mirad el panel de pasos: eso es el agente haciendo su trabajo.»
Alguien aprueba un aviso sin leer	«Para. Léelo en voz alta. ¿Qué le estás dejando hacer?»
El agente hace algo mal delante de todos	«Perfecto, esto es material didáctico. Mirad los pasos y decidme en cuál se ha ido.»
Preguntan algo que no sabes	«No lo sé, lo apunto y te lo contesto.» Apúntalo de verdad.
Se cae la red	«Pasamos al plan B: lo hago yo en pantalla y vosotros vais siguiendo.»
Alguien quiere subir los permisos al máximo	«Antes de eso, dime qué tarea es. Casi siempre se arregla moviendo la carpeta.»
El grupo va muy rápido	«Vale, entonces me adelanto: ¿qué creéis que pasa si dos personas trabajan en la misma carpeta a la vez?»